UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Escola Tècnica Superior d'Enginyeria
de Telecomunicació de Barcelona

Prompting in Deep Learning Speech Recognition

Masters Thesis submitted to the Faculty of the
Escola Tècnica d'Enginyeria de Telecomunicació de Barcelona
Universitat Politècnica de Catalunya
by

Lucas Takanori Sanchez Shiromizu

In partial fulfillment of the requirements for the master in
Master's degree in Advanced Telecommunication Technologies (MATT), track in Deep Learning for Multimedia Processing

Director: Dr Francisco Javier Hernando Pericas
Barcelona, October 22nd 2024

Contents

List of Figures

List of Tables

Revision history and approval record

Revision	Date	Purpose
0	06/06/2024	Document creation
1	13/06/2024	Document revision
2	22/10/2024	Document final revision

DOCUMENT DISTRIBUTION LIST

Name	e-mail
Lucas Takanori Sanchez Shiromizu	lucas.takanori.sanchez@estudiantat.upc.edu
Francisco Javier Hernando Pericas	javier.hernando@upc.edu

Written by:		Reviewed and approved by:	
Date	22/10/2024	Date	22/10/2024
Name	Lucas Takanori Sanchez Shiromizu	Name	Javier Hernando Pericas
Position	Project Author	Position	Project Supervisor

Abstract

Adapting large-scale pre-trained automatic speech recognition (ASR) models, such as OpenAI's Whisper, to specific tasks or languages remains a challenging problem due to the substantial computational resources required for traditional fine-tuning methods. This limitation is particularly significant in real-world scenarios where resources are constrained, and efficient adaptation is essential for handling diverse languages and domains.

To address this issue, this thesis explores two parameter-efficient fine-tuning (PEFT) techniques: soft prompting and Low-Rank Adaptation (LoRA). Soft prompting leverages trainable prompt embeddings to adapt the model with minimal parameter updates, while LoRA applies low-rank transformations to the model's weight matrices, reducing the number of trainable parameters. Through experiments on the 3CatParla dataset for Catalan speech recognition, we demonstrate that these techniques achieve competitive performance with significantly lower computational demands. LoRA, in particular, shows strong results in terms of efficiency and accuracy, while soft prompting exhibits performance limitations with larger models. This work opens pathways for further research into hybrid methods and evaluation across more diverse datasets, contributing to the field of efficient ASR adaptation for low-resource environments.

1 Introduction

The field of ASR has undergone significant evolution over the past few decades. Initially, traditional models such as HMM and GMM were the primary approaches used in ASR systems [5]. These statistical models provided a foundation for early speech recognition tasks but faced limitations in handling complex scenarios, particularly in noisy environments [6].

As computational power increased and more data became available, researchers began exploring neural network-based approaches. The introduction of deep learning techniques in the early 2010s marked a turning point in ASR technology [7]. DNN demonstrated superior performance in modeling both temporal and spectral features from raw audio data, addressing many of the shortcomings of traditional methods [8].

The advance of specialized neural network architectures, such as CNN and RNN, further advanced the field. These architectures proved particularly adept at capturing the nuances of audio data, which are crucial for improving ASR accuracy [9]. LSTM networks, a type of RNN, became especially popular due to their ability to model long-term dependencies in speech [10].

In recent years, the emergence of Transformer-based models has revolutionized ASR systems. Models like Whisper, developed by OpenAI, utilize Transformer architectures to handle large-scale speech recognition tasks across multiple languages while significantly improving transcription accuracy [1]. These models excel at processing both short- and long-term dependencies in speech, making them ideal for tasks like transcription and translation.

Despite the impressive performance of large models, challenges remain in optimizing these systems to run efficiently, especially when dealing with low-resource languages or domain-specific data [11]. This thesis focuses on exploring and comparing different adaptation techniques for the Whisper model, a state-of-the-art ASR system, to address these challenges and further improve its performance across various domains and languages.

To address these challenges, this thesis explores PEFT techniques [12] such as LoRA [13] and soft prompting [14]. These methods aim to adapt large pre-trained models like Whisper to specific domains or languages with minimal computational overhead. By focusing on updating only a small subset of parameters or introducing trainable prompts, these techniques promise to reduce the resources required for adaptation while maintaining model performance. This research seeks to compare the effectiveness of these approaches in the context of ASR, particularly for adapting Whisper to the Catalan language, and to analyze the trade-offs between adaptation performance, computational efficiency, and resource requirements.

1.1 Motivation and Objectives

The field of ASR has seen remarkable progress with the developement of large-scale models like Whisper [1]. These models have demonstrated unprecedented performance across various languages and domains, pushing the boundaries of what's possible in speech recog-

nition. However, this progress comes with significant challenges that form the core motivation for this research.

The trend towards larger models is driven by their ability to capture complex patterns and generalize across diverse tasks. As model size increases, we observe improved accuracy, robustness, and the capacity to handle multiple languages and accents. This scalability is particularly valuable in ASR, where the ability to understand diverse speech patterns is crucial.

However, the growth in model size leads to several critical issues. Training and deploying large models require substantial computational power, often only available to well-resourced organizations. This creates a barrier for smaller research groups or companies. Large models, while general, may not perform optimally on specific domains or less-represented languages without fine-tuning [15]. Traditional fine-tuning methods often require substantial data and computational resources.

The primary motivation for this research stems from the need to address these challenges, making large-scale ASR models like Whisper more flexible, efficient, and applicable to diverse real-world scenarios. As ASR technology continues to evolve, there is a growing demand for systems that can quickly adapt to new languages, accents, or specialized vocabularies without the need for extensive retraining or the computational resources typically associated with fine-tuning large models.

This thesis aims to tackle the fundamental problem of efficiently adapting large-scale ASR models to specific tasks or languages while minimizing computational costs and resource requirements. By exploring PEFT techniques such as LoRA [13] and soft prompting [14], we seek to develop methods that allow for rapid and resource-efficient adaptation of models like Whisper. This approach could make languages or domains with limited data and computational resources more accessible.

To address these motivations, the key objectives of this thesis are:

1. To explore and evaluate different ASR model adaptation techniques, such as fine-tuning, LoRA and soft prompting.
2. To compare the effectiveness of soft prompting and LoRA in adapting the Whisper model for specific tasks, such as Catalan language recognition.
3. To analyze the trade-offs between model performance, parameter efficiency, and computational cost when applying PEFT techniques.

While the main objectives are focused on the exploration of different ASR model adaptation techniques, the side objectives of this work are:

1. To gain an in-depth understanding of the Whisper model architecture and its components.
2. To develop skills in using high-performance computing clusters for training and adapting large-scale ASR models.
3. To contribute to the development of flexible ASR models that can be efficiently

adapted to specialized domains or low-resource languages.

4. To collaborate with the Barcelona Supercomputing Center (BSC) and leverage their resources for high-performance computing, contributing to projects aimed at preserving the Catalan language.
5. To learn and apply best practices in managing and processing large-scale datasets, including data preprocessing, cleaning, and splitting for machine learning tasks.
6. To develop proficiency in utilizing experiment tracking tools, such as Weights & Biases, for effective monitoring and comparison of model training and adaptation experiments.

This research builds upon the Whisper model developed by Radford et al. [1] and extends it by exploring various adaptation techniques. The implementation will utilize the open-source Whisper model and APIs provided by Huggingface, along with custom scripts for data processing and model evaluation.

By investigating different adaptation techniques, this research aims to contribute to the broader field of ASR, potentially uncovering insights that could be applied to other languages or speech recognition tasks. Improving the development and adaptation of new ASR models for Catalan can have numerous real-world applications, from enhancing accessibility for Catalan speakers to supporting language preservation efforts.

By achieving these objectives, this thesis aims to influence the broader field of speech recognition by offering insights into how large ASR models can be efficiently adapted to specific tasks or languages. Moreover, it will contribute directly to the promotion and preservation of the Catalan language through the development of cutting-edge ASR technology, supporting societal efforts to sustain linguistic diversity in the digital world.

1.2 Workplan

The implementation of a structured workplan was essential for achieving the objectives of this thesis. The project workflow can be divided into several key stages:

1. **Literature Review** (Weeks 1-3):
    - Comprehensive review of current literature on ASR, with a focus on adaptation techniques for large language models.
    - In-depth study of the Whisper model architecture and its applications.
    - Exploration of fine-tuning and soft prompting techniques in the context of ASR.
2. **Data Preparation** (Weeks 4-6):
    - Acquisition and preprocessing of Catalan speech datasets.
    - Data cleaning, normalization, and formatting for compatibility with the Whisper model.
    - Creation of train, validation, and test splits.
3. **Baseline Model Implementation** (Weeks 7-8):
    - Setting up the Whisper model for Catalan ASR.
    - Establishing baseline performance metrics on the Catalan dataset.
4. **Fine-tuning Implementation** (Weeks 9-11):
    - Implementation of fine-tuning procedures for the Whisper model.
    - Experimentation with different fine-tuning strategies and hyperparameters.
    - Performance evaluation and comparison with the baseline.
5. **Soft Prompting Implementation** (Weeks 12-14):
    - Development and implementation of soft prompting techniques for Whisper.
    - Experimentation with various soft prompt configurations.
    - Performance evaluation and comparison with fine-tuning and baseline results.
6. **Soft Prompting Implementation** (Weeks 15-17):
    - Development and implementation of soft prompting techniques for Whisper.
    - Experimentation with various soft prompt configurations.
    - Performance evaluation and comparison with fine-tuning and baseline results.
7. **Comparative Analysis** (Weeks 17-18):

- Comprehensive comparison of baseline, fine-tuning, soft prompting and LoRA approaches.
- Analysis of trade-offs between performance, computational efficiency, and parameter efficiency.
- Identification of the most effective adaptation technique(s) for Catalan ASR.

8. **Thesis Writing and Revision** (Weeks 18-21):
   - Compilation of results and findings into a coherent thesis document.
   - Critical analysis and discussion of the implications of the research.
   - Revision and refinement of the thesis based on feedback.

This workplan provides a structured approach to addressing the research objectives, allowing for systematic implementation and evaluation of different adaptation techniques for the Whisper model in Catalan ASR.

2 State of the art

2.1 Traditional Automatic Speech Recognition Systems

Automatic Speech Recognition (ASR) has evolved significantly over the years. Before the introduction of deep learning, ASR systems primarily relied on traditional machine learning techniques and statistical models. The typical ASR pipeline has the following components:

The initial stage of feature extraction converts the raw audio signal into a more compact and informative representation. Common techniques in this stage include Mel-Frequency Cepstral Coefficients (MFCCs) [16] and Linear Predictive Coding (LPC) [17]. The recognizer component integrates several models to convert acoustic features into text. It includes an acoustic model, which models the relationship between audio features and linguistic units (e.g., phonemes), traditionally implemented using Hidden Markov Models (HMMs) and Gaussian Mixture Models (GMMs) [5].

A pronunciation model, or lexicon, serves as a dictionary mapping words to their phonetic representations. The language model provides the probability of word sequences, using approaches like N-gram models and statistical language models [18]. The decoder integrates information from the recognizer to determine the most likely sequence of words. It typically employs search algorithms such as the Viterbi algorithm [19] and beam search [20].

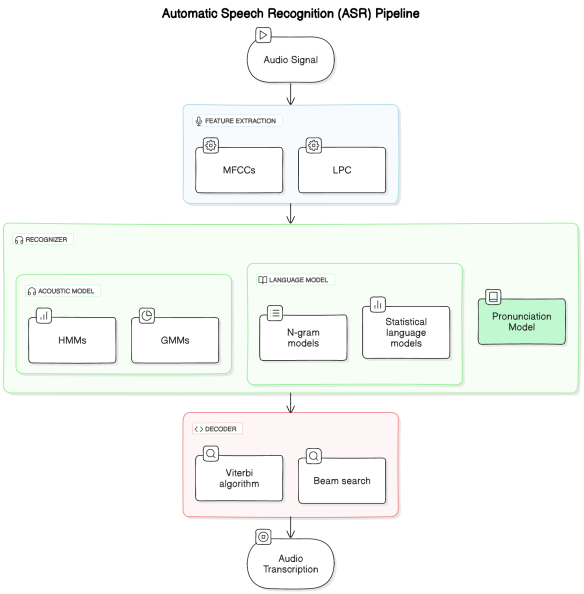

Figure 1: Automatic Speech Recognition Pipeline

These components work together to convert speech input into text output. The system takes in the speech signal, extracts relevant features, and passes them to the recognizer. The recognizer uses its acoustic model to determine likely phonemes, consults the pro-

nunciation model to map phonemes to words, and applies the language model to find the most probable sequence of words. The decoder orchestrates this process to produce the final transcription.

While this traditional framework has been successful, it has limitations in handling variability in speech, noise robustness, and scaling to large vocabularies. These challenges have motivated the shift towards deep learning approaches, which we will explore in the following section.

2.2 Deep Learning for Automatic Speech Recognition

Deep learning has revolutionized Automatic Speech Recognition (ASR) by significantly improving accuracy and robustness. Various neural network architectures and techniques have been developed to enhance ASR systems. This section provides an overview of key deep learning techniques and their evolution in ASR.

The deep learning era in ASR began with the introduction of Deep Neural Networks (DNNs), which replaced traditional Gaussian Mixture Models (GMMs) for acoustic modeling. DNNs, with their multiple layers of neurons, provided superior feature representation and improved recognition accuracy by capturing complex patterns in speech data [21]. Building upon this foundation, researchers integrated Convolutional Neural Networks (CNNs) into ASR systems. CNNs introduced convolutional layers that process input speech signals hierarchically, capturing local temporal and spectral features. This innovation made ASR systems more effective in handling variations in speech signals, such as accents and background noise [9].

As the field progressed, the focus shifted to better modeling the temporal dynamics of speech. This led to the adoption of Recurrent Neural Networks (RNNs) and their variants, such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU). These architectures excel at capturing temporal dependencies in sequential data, making them well-suited for ASR tasks. LSTM and GRU networks, in particular, addressed the vanishing gradient problem, allowing the models to learn long-term dependencies in speech patterns [10].

The next significant leap came with the introduction of Sequence-to-Sequence (Seq2Seq) models, including the groundbreaking Transformer architecture. These models directly map input speech sequences to output text sequences, utilizing an encoder-decoder structure with attention mechanisms. This approach enabled ASR systems to handle long-range dependencies more effectively, resulting in improved transcription accuracy. The Transformer's ability to process entire input sequences in parallel marked a particular breakthrough in ASR performance [22].

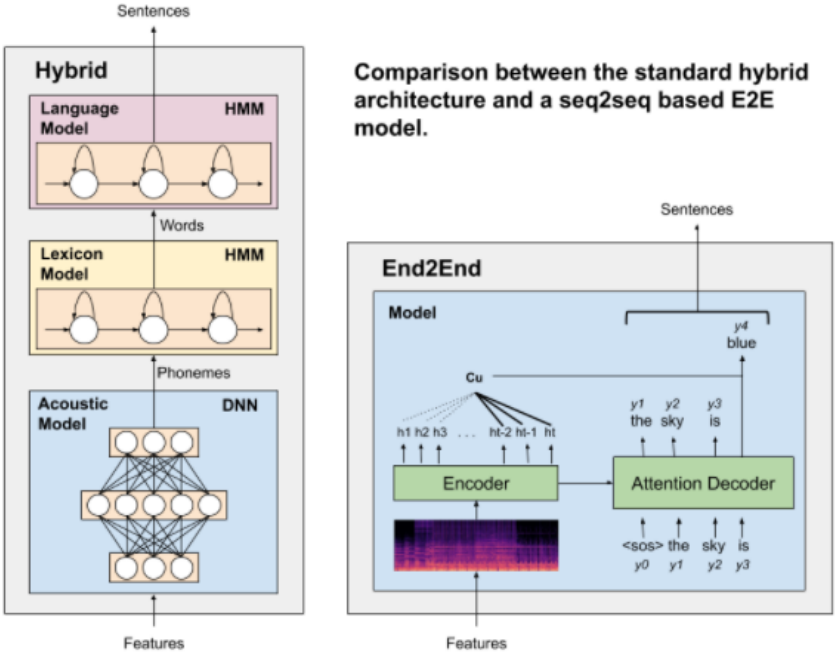

Figure 2: Comparison between the standard hybrid architecture and a seq2seq based E2E model. Source: verbio.com

The pursuit of simplification led to the development of End-to-End (E2E) models. These models integrated the entire ASR pipeline into a single neural network, reducing the need for handcrafted features and simplifying the training process. True E2E models learn to map speech inputs directly to text outputs without explicit intermediate representations. Examples include the Listen, Attend and Spell (LAS) model [23] and the Recurrent Neural Network Transducer (RNN-T) [24].

To facilitate the training of these E2E models, various techniques were developed to align output tokens with audio inputs. Connectionist Temporal Classification (CTC) [25] allows the model to learn this alignment automatically during training. The Recurrent Neural Aligner (RNA) [26] offers an alternative approach to learning alignments. These alignment techniques, while not E2E models themselves, have been crucial in enabling the development and success of E2E ASR systems.

Further refinement of these concepts resulted in advanced architectures like the Conformer Transducer. This model combined the strengths of CNNs and self-attention mechanisms, capturing both local and global dependencies in speech signals. By integrating convolutional layers and transformer-based self-attention within a single block, the Conformer Transducer enhanced temporal pattern recognition and context modeling, leading to improved performance in challenging environments and diverse speaking styles [27].

The field then saw a shift towards leveraging large amounts of unlabeled data through self-supervised learning techniques. Methods like wav2vec and HuBERT pre-train models on vast amounts of unlabeled speech data before fine-tuning on labeled datasets. This

approach significantly reduced the need for labeled data and enhanced ASR performance, particularly for low-resource languages and domains [28, 29].

This continuous evolution of deep learning techniques has propelled ASR systems to new heights of accuracy, robustness, and versatility. Each advancement has built upon the strengths of previous approaches while addressing their limitations, resulting in increasingly sophisticated and effective ASR systems. As research continues, future developments are likely to focus on further improving these models, addressing current limitations, and expanding their applicability to an even broader range of languages and environments.

The Whisper model, developed by OpenAI, represents a significant advancement in automatic speech recognition (ASR) technology. Introduced by Radford et al. [1], Whisper has demonstrated remarkable performance across various languages and tasks, setting new benchmarks in the field of speech processing. Its architecture combines several innovative features that contribute to its effectiveness in handling diverse speech recognition challenges. In the following section, we'll delve into the details of Whisper's model architecture, exploring the key components that make it a powerful tool for ASR.

Model Architecture

At its core, Whisper is based on a sequence-to-sequence encoder-decoder architecture, drawing inspiration from the Transformer model introduced by Vaswani et al. [19]. However, Whisper incorporates several modifications to adapt this architecture specifically for speech recognition tasks.

The encoder in Whisper is responsible for processing the input audio features. The audio is converted into 80-channel log-mel spectrograms, computed on 25ms windows with a 10ms step. This log-mel spectrogram is feed to two convolution layers with a filter width of 3 and the GELU activation function is applied. The second convolution layer has a stride of two, effectively downsampling the input, sinusoidal position embeddings are added to the output of the convolutional stem. This layer is followed by multiple Transformer blocks. The use of Transformer blocks allows the model to attend to different parts of the input sequence, capturing both short-term and long-term dependencies in the speech signal.

The decoder, on the other hand, is tasked with generating the output text sequence. It consists of Transformer blocks that utilize both causal self-attention and cross-attention mechanisms. The causal self-attention ensures that the model only attends to previous tokens during the generation process, maintaining the sequential nature of the output. The cross-attention mechanism allows the decoder to focus on relevant parts of the encoded input while generating each output. Finally the sequence is feed to a linear layer followed by a softmax to predict the output token.

The model processes this input through its encoder-decoder architecture as follows:

$$\mathbf{H} = \text{AudioEncoder}(\mathbf{X}) \tag{1}$$

$$\hat{y}_t = \text{TextDecoder}(p, \hat{y}_{1:t-1}, \mathbf{H}) \tag{2}$$

where H represents the hidden representations from the encoder, X is the 80-dimensional log-Mel spectrogram of 30 seconds, p are special prompts, and $\hat{y}_t$ is the output token at time t [1].

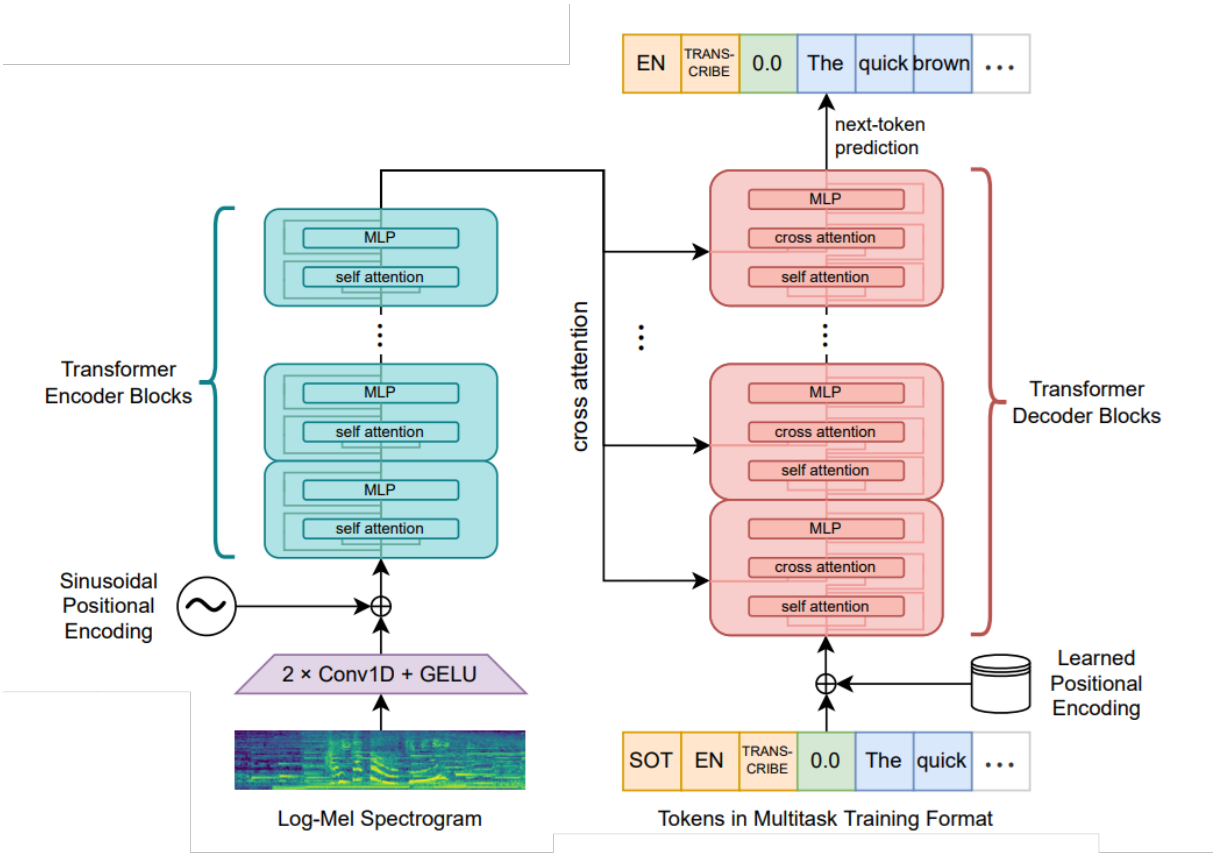

Figure 3: Whisper model representation from [1]

The decoder in Whisper is designed to be flexible, allowing for various speech processing tasks through the use of prompt engineering. This approach, often referred to as "hard prompting," involves carefully crafting input sequences to guide the model's behavior without modifying its parameters.

The decoder input sequence typically consists of several components:

- A special start token: `<|startoftranscript|>`
- A language token: e.g., `<|en|>` for English
- A task specifier: e.g., `<|transcribe|>` or `<|translate|>`
- An optional timestamp token: `<|notimestamps|>` if timestamps are not required
- Previous context (if any)

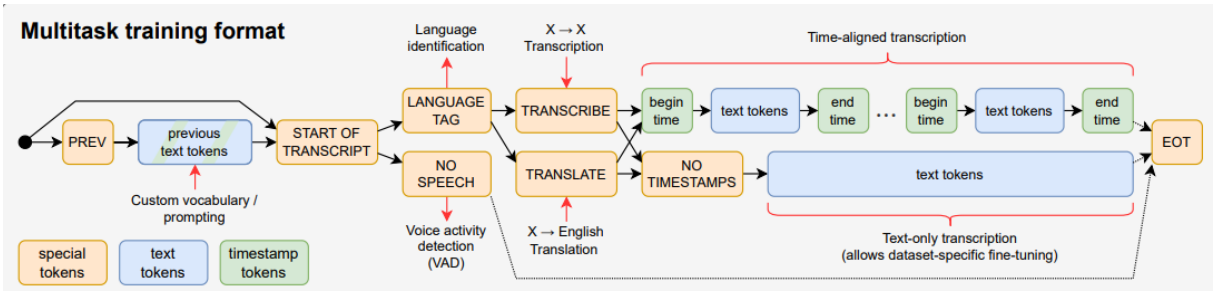

Figure 4: Decoder input token structure from [1]

For example, a typical decoder input for English transcription might look like this: `<|startoftranscript|><|en|><|transcribe|><|notimestamps|>`

The decoder processes this input sequence along with the encoder output to generate the final output. The output is produced token by token, with each token being selected based on the highest probability from the model's softmax layer. The dimensionality of this output layer corresponds to the model's vocabulary size, which in Whisper's case is 51,865 tokens.

Previous Token and Previous Text As seen in figure 4, the decoder input can also include a "prev token" and "previous text" tokens. These components play crucial roles in the model's ability to handle long-form audio and maintain context:

1. Previous Token (prev token): This token represents the last token generated in the previous decoding step. It allows the model to maintain continuity in its output, especially when processing long audio streams that are split into multiple segments.

2. Previous Text: This refers to a portion of the previously transcribed text, which is fed back into the decoder as part of its input. This mechanism serves several purposes: - It provides context for the current transcription, helping the model maintain consistency across segments. - It allows the model to potentially correct or refine previous outputs based on new information. - In long-form transcription, it helps the model keep track of the overall discourse and maintain topical coherence.

The inclusion of previous text is particularly important when transcribing long audio files that exceed the model's maximum context window (typically 30 seconds). By providing the model with the tail end of the previous transcription, it can better handle speaker continuity, contextual references, and maintain a coherent narrative across segment boundaries.

During training, Whisper randomly includes previous text with some probability, allowing the model to learn how to effectively use this additional context. In inference, the amount of previous text included can be adjusted based on the specific requirements of the task at hand.

The use of special tokens in the decoder input allows for fine-grained control over the model's behavior. For instance, by changing the language token, one can perform zero-

shot cross-lingual ASR or translation. By modifying the task specifier, the same model can be used for transcription, translation, or language identification tasks.

Interestingly, Radford et al. [1] found that using `<|transcribe|>` instead of `<|translate|>` led to better performance for speech translation (ST) tasks, despite Whisper being originally trained with `<|translate|>` for translation tasks. This counterintuitive result highlights the importance of experimentation in prompt engineering.

The output generation process continues until the model produces an end-of-transcript token `<|endoftranscript|>`. For tasks requiring timestamps, the model can be prompted to interleave timestamp tokens with the output text, providing time-aligned transcriptions.

Training Process

The training process of Whisper is notable for its scale and diversity, which contribute significantly to the model's robustness and generalization capabilities. Radford et al. [1] trained Whisper on an extensive dataset comprising 680,000 hours of multilingual and multitask supervised data collected from the web. This dataset is remarkably diverse, including a wide range of accents, background noises, and recording conditions, which helps the model generalize to real-world scenarios.

The multilingual aspect of Whisper's training is equally important. The model is trained on 98 languages, which allows for effective cross-lingual transfer. This means that Whisper can perform well even on languages it hasn't seen during training, a capability known as zero-shot generalization.

Interestingly, unlike many contemporary ASR systems, Whisper does not rely on unsupervised pre-training. Instead, it leverages large-scale supervised training on diverse data. This approach has proven effective, allowing Whisper to achieve state-of-the-art performance without the need for separate pre-training and fine-tuning stages.

Limitations and Future Directions

Despite its impressive capabilities, Whisper does have some limitations that are important to consider. The larger versions of the model, while more accurate, require significant computational resources for real-time processing. This can be a constraint in resource-limited environments or for applications requiring low latency.

Like many large language models, Whisper can sometimes produce "hallucinations" - plausible-sounding but incorrect transcriptions. This is particularly noticeable with the larger model variants. While this issue is not unique to Whisper, it highlights the ongoing challenge in ASR of balancing fluency with accuracy.

As with all large-scale machine learning models trained on web-scraped data, there's also the potential for biases present in the training data to be reflected in the model's outputs. This underscores the importance of careful data curation and ongoing efforts to address and mitigate biases in AI systems.

In conclusion, the Whisper model represents a significant advancement in speech recognition technology. Its architecture, training process, and resulting capabilities demonstrate the potential of large-scale, multitask, multilingual training in creating robust and versatile speech processing systems. As research in this field continues, future work may focus on addressing the noted limitations, further improving performance on low-resource languages, and exploring ways to reduce the computational requirements of these powerful models.

2.3 Model Adaptation

In the field of automatic speech recognition (ASR), model adaptation techniques play a crucial role in improving the performance of pre-trained models on specific domains or tasks. These techniques are essential as pre-trained models are typically trained on general-purpose datasets that may not fully cover domain-specific vocabularies, accents, or acoustic environments. This section explores three model adaptation methods: fine-tuning, Low-Rank Adaptation (LoRA) and Soft Prompting. Each method provides unique advantages and trade-offs in terms of performance, computational cost, and flexibility, allowing practitioners to tailor ASR systems to the needs of different tasks.

2.3.1 Fine-tuning

Fine-tuning involves additional training of a pre-trained model on a domain-specific dataset to adapt its parameters to the target task. The fundamental idea behind fine-tuning is that a pre-trained model serves as a good initialization point, and by updating its parameters on a smaller, task-specific dataset, it can better fit the characteristics of the target domain while retaining the general knowledge obtained during the initial pre-training [30]. The process of fine-tuning in ASR models such as Whisper generally follows these steps:

1. Preparing a domain-specific dataset $\mathcal{D}_{\text{target}}$
2. Initializing the model with pre-trained weights $\theta_{\text{pretrained}}$
3. Training the model on the new dataset to obtain fine-tuned weights $\theta_{\text{fine-tuned}}$
4. Evaluating the fine-tuned model on a held-out test set

Mathematically, this process can be described as minimizing a task-specific loss function $\mathcal{L}_{\text{task}}$ over the target dataset $\mathcal{D}_{\text{target}}$:

$$\theta_{\text{fine-tuned}} = \arg\min_{\theta} \mathbb{E}_{(x,y)\in\mathcal{D}_{\text{target}}} \left[\mathcal{L}_{\text{task}}(f_\theta(x), y)\right]$$

where $f_\theta(x)$ represents the model's prediction for input x with parameters θ. The model initially starts with $\theta_{\text{pretrained}}$, and gradient descent is used to update the parameters toward $\theta_{\text{fine-tuned}}$.

Fine-tuning techniques can be further categorized into full fine-tuning and partial fine-tuning [31]. In full fine-tuning, all model parameters are updated, which can be highly

effective when the target domain differs significantly from the pre-training domain. However, full fine-tuning is computationally expensive, especially for large models like Whisper, and risks overfitting when the target dataset is small [32].

On the other hand, partial fine-tuning updates only a subset of the model's parameters, typically those in the higher layers of the network, which capture more task-specific features [31]. This approach is computationally cheaper and can prevent overfitting but may not yield as significant performance improvements as full fine-tuning. The selection of layers to fine-tune depends on the similarity between the pre-training and target tasks, as well as the available resources.

While fine-tuning can significantly improve performance on specific tasks, it may become impractical for very large models due to the computational cost of maintaining multiple versions of the model. This challenge has been highlighted by Ma et al. [2], who emphasized that large models like Whisper face scalability issues as the number of tasks and domains increases.

2.3.2 Low-Rank Adaptation (LoRA)

To address the computational challenges associated with fine-tuning large models, parameter-efficient fine-tuning (PEFT) techniques have been developed. One such technique is Low-Rank Adaptation (LoRA), introduced by Hu et al. [13]. LoRA is based on the observation that the updates to the model's weights during fine-tuning often lie in a low-rank subspace. By restricting weight updates to this subspace, LoRA significantly reduces the number of trainable parameters without sacrificing performance.

The core idea behind LoRA is to decompose the weight updates into two smaller matrices:

$$\Delta W = AB^T$$

where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{d \times r}$ are trainable matrices, and $r \ll d$ is the rank of the decomposition. This low-rank decomposition significantly reduces the number of parameters that need to be updated during training. Instead of fine-tuning the entire weight matrix $W \in \mathbb{R}^{d \times d}$, LoRA only updates the small matrices A and B, while the pre-trained weight matrix $W_{\text{pretrained}}$ remains frozen. The modified weight matrix during fine-tuning becomes:

$$W_{\text{adapted}} = W_{\text{pretrained}} + AB^T$$

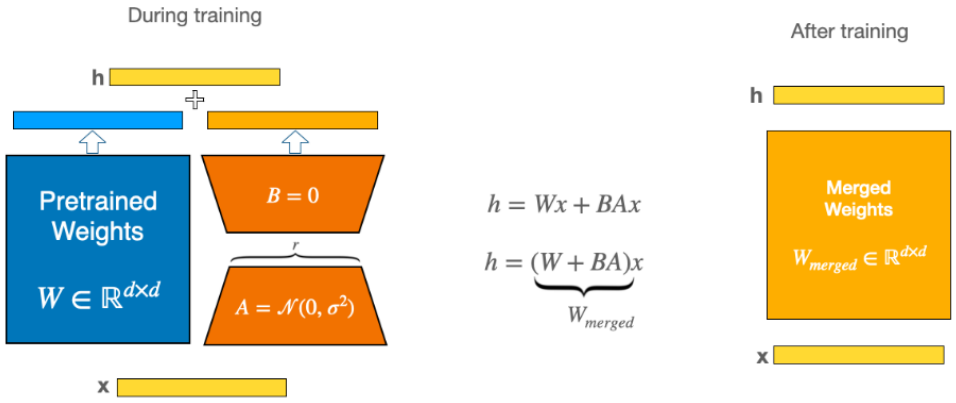

Figure 5: LoRA during training and inference.

During training (left), the pretrained weights W remain frozen while low-rank matrices A and B are trained, with the outputs summed. After training (right), the matrices can be merged into a single weight matrix W_{merged} for efficient inference.

LoRA offers several advantages over traditional fine-tuning:

- **Parameter Efficiency:** By updating only a small subset of parameters (those in A and B), LoRA significantly reduces the computational and storage costs associated with fine-tuning.
- **Flexibility:** LoRA allows for rapid adaptation to multiple domains without modifying the base model. Multiple low-rank adaptations can be stored and applied as needed.
- **Performance:** Despite its efficiency, LoRA achieves performance comparable to full fine-tuning in many tasks, as shown in experiments by Hu et al. [13].

In the context of ASR, LoRA has proven particularly useful for adapting large models like Whisper to specific accents, languages, or domains, where only a small number of parameters need to be adjusted for improved performance. For instance, Pasad et al. [33] demonstrated that LoRA could effectively adapt ASR models to various noisy environments and speakers without the need for full fine-tuning.

However, the effectiveness of LoRA may vary depending on the specific task and architecture. Certain tasks might require a higher rank in the decomposition to capture the nuances of the target domain, while others might benefit from a lower rank to preserve computational efficiency. Research is ongoing to optimize LoRA and other PEFT methods for different ASR applications [34].

To understand the parameter reduction achieved by LoRA, consider the dimensions involved. For a typical weight matrix $W \in \mathbb{R}^{d \times d}$, the total number of parameters is d^2. With LoRA, the number of parameters is reduced to $2rd$ (since $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{d \times r}$), which represents a significant reduction when $r \ll d$. The ratio of trainable parameters in LoRA compared to full fine-tuning is:

$$\frac{2rd}{d^2} = \frac{2r}{d}$$

Thus, the parameter savings scale inversely with the rank r. In practice, r is often much smaller than d, making LoRA a highly efficient adaptation method.

2.3.3 Soft prompting

Soft prompting, also known as prompt tuning or continuous prompting, is a more recent technique that combines elements of both hard prompting and fine-tuning [35]. It involves learning continuous prompt embeddings that are prepended to the input sequence during inference.

It was developed to address limitations of discrete hard prompts, particularly their lack of trainability and limited expressiveness [35]. While hard prompts are restricted to the model's vocabulary, continuous soft prompts allow for learning optimal prompting vectors through backpropagation furthermore they allow us to capture task-specific information in continuous space. They provide a more efficient parameter usage compared to full fine-tuning.

Given a log-Mel spectrogram X of speech, soft prompts extend the model to transcribe the text token sequence $\hat{y}$:

$$H = \text{AudioEncoder}_{\phi_e}([P_e, \text{Conv}(X)]) \tag{3}$$

$$\hat{y}_t = \text{TextDecoder}_{\phi_d}(P_d, \hat{y}_{1:t-1}, H) \tag{4}$$

where $P_e \in \mathbb{R}^{d_m \times L_e}$ and $P_d \in \mathbb{R}^{d_m \times L_d}$ are learnable soft prompts, L_e and L_d are hyperparameters denoting the number of soft prompts [1].

During training, we minimize the cross-entropy loss L_{CE} between model predictions $\hat{y}$ and ground-truth transcriptions y:

$$P^* = \arg\min_P L_{CE}(\hat{y}, y) \tag{5}$$

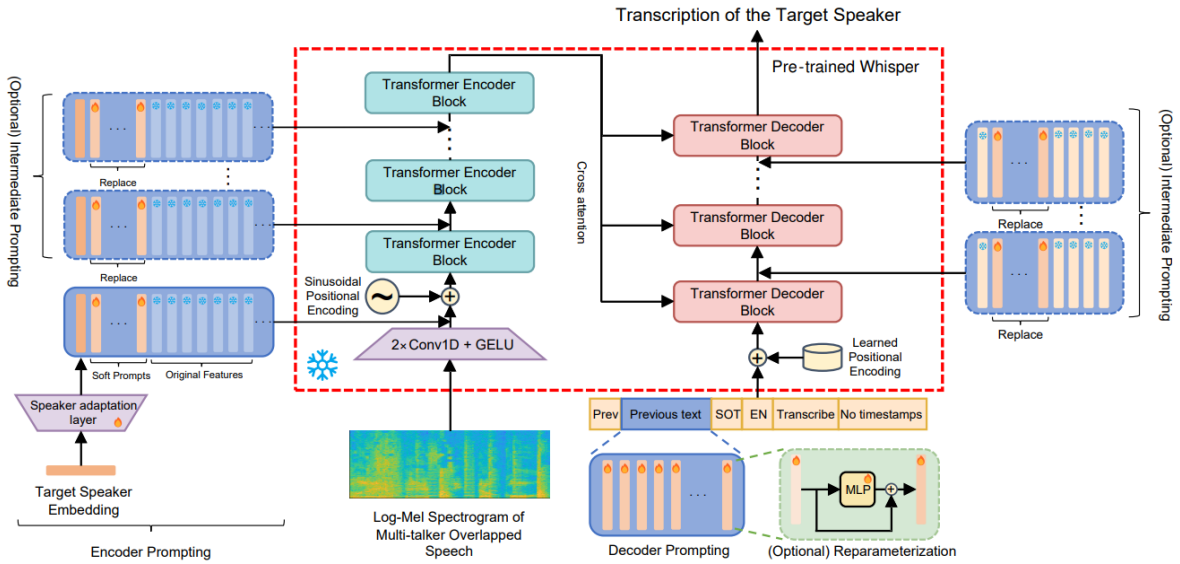

Figure 6: Overview of the soft prompting framework for TS-ASR used in [2].

In figure 6 Modules with solid-line borders represent the base pre-trained Whisper model. Soft prompts can be added at different locations: encoder prompting (left), decoder prompting (right), and optionally with intermediate prompting throughout the model. The reparameterization module (bottom right) can be optionally added to improve training stability. The pre-trained Whisper processes the log-Mel spectrogram input through convolution layers and transformer blocks, with the decoder utilizing learned positional encodings and cross-attention mechanisms

1. **Encoder-only soft prompting**: In this approach, learnable prompt embeddings are prepended only to the input sequence processed by the encoder. This can be particularly effective for tasks that primarily require adaptation of the audio processing part of the model.

2. **Decoder-only soft prompting**: Here, the learnable prompts are added only to the beginning of the decoder input sequence. This approach can be useful for adapting the language modeling capabilities of Whisper without modifying its audio processing.

3. **Encoder-decoder soft prompting**: This comprehensive approach involves adding separate learnable prompts to both the encoder and decoder inputs. It allows for adaptation of both the audio processing and language modeling components of Whisper.

In the context of Whisper, soft prompting implementation is done by:

1. Initializing a set of learnable prompt embeddings
2. Freezing the main model parameters
3. Training only the prompt embeddings on a target dataset
4. Prepending the learned embeddings to input sequences during inference

Lester et al. [35] showed that as language models scale to billions of parameters, the gap in performance between soft prompting and full fine-tuning becomes negligible. This finding suggests that soft prompting could be an increasingly viable alternative to fine-tuning as models continue to grow in size.

3 Methodology

In this section, we are going to discuss the approach that has been followed to implement the soft prompts on to Whisper model

3.1 Working Environment

This project was developed using the MareNostrum 5 supercomputer, specifically utilizing one node from the Accelerated Partition (ACC). The ACC nodes of MareNostrum 5 provide a powerful environment for deep learning tasks, particularly suitable for our experiments with the Whisper model.

3.1.1 Hardware Specifications

Each ACC node in MareNostrum 5 is equipped with:

- 2x Intel Xeon Platinum 8460Y+ processors, each with 40 cores running at 2.3GHz, providing a total of 80 CPU cores per node
- 4x NVIDIA Hopper H100 GPUs, each with 64GB of HBM2 memory
- 512GB of main memory (16x 32GB DDR5 DIMMs running at 4800MHz)
- 480GB NVMe local storage
- 4x ConnectX-7 NDR200 InfiniBand network interfaces, offering a total bandwidth of 800Gb/s per node

This hardware configuration provides substantial computational power, particularly from the four NVIDIA H100 GPUs, which are well-suited for training and fine-tuning large language models like Whisper.

3.1.2 Software Environment

The development environment was set up on the Linux operating system provided by MareNostrum 5. To ensure a clean and reproducible environment, we utilized Python's virtual environment (venv) feature. This allowed us to isolate our project dependencies and avoid conflicts with system-wide packages.

For our experiments, we used the Whisper model implementation provided by Hugging Face's Transformers library. This implementation offers a convenient and well-documented interface to work with the Whisper model, allowing for easy experimentation with various model sizes and adaptation techniques.

To track all system and variable metrics throughout our experiments, we integrated Weights & Biases (wandb) into our workflow. Wandb provided us with a robust platform for experiment tracking, visualization, and collaboration. It allowed us to monitor key performance indicators, system utilization, and model metrics in real-time, facilitating easier comparison between different experimental runs and configurations.

The project's codebase was version-controlled using Git and hosted on GitHub at `https://github.com/LucasTakanori/SoftPrompting`. This repository contains all the necessary code for reproducing our experiments, including scripts for data preprocessing, model training, and evaluation. It also includes configuration files for Wandb integration, ensuring that our experiment tracking setup can be easily replicated.

3.2 Dataset

For our experiments, we utilized the 3CatParla ASR corpus, a comprehensive manually annotated dataset of broadcast Catalan TV shows. The corpus consists of 218,532 speech recordings, totaling 731 hours and 21 minutes of audio content. Due to the extensive transcription process involving multiple annotators, the corpus is divided into six categories based on transcription quality:

- Perfect matches (90h00m47s, 32,287 files)
- Few typos (157h16m45s, 47,803 files)
- Below 3% CER (215h30m08s, 60,910 files)
- Around 7% CER (146h26m06s, 41,322 files)
- Below 15% CER (101h39m58s, 29,305 files)
- Word count mismatch (11h31m09s, 3,721 files)

Additionally, the corpus includes development (dev) and test splits, each comprising approximately 4 hours and 28 minutes of audio, selected from the highest quality "perfect matches" category.

The quality of transcriptions was determined through a ranking process using various ASR systems such as NeMo, Wav2Vec, and Whisper. This approach ensured a diverse range of speech data quality, allowing us to evaluate our models' performance across different levels of transcription accuracy.

The 3CatParla "Perfect matches" subset was used for the training of the 3 model adaptation methods, with a focus on the high-quality training data. We made the deliberate choice to use only the "perfect matches" subset based on several important factors:

Transcription Quality: The "perfect matches" subset, comprising 90 hours and 47 seconds of audio (32,287 files), represents the highest quality transcriptions in the corpus. These transcriptions were verified to have perfect alignment with the audio content, ensuring the highest level of accuracy in our training data.

Minimizing errors in Training Data: By focusing on perfect matches, we significantly reduce the potential for introducing errors into our training process. Lower quality transcriptions, even those with few typos, could potentially lead to the model learning incorrect

Baseline Establishment: Using the highest quality subset provides a strong baseline for our experiments. This approach allows us to assess the model's performance under ideal conditions before potentially expanding to include data of varying quality in future iterations.

Resource Efficiency: With 90 hours of high-quality audio, the perfect matches subset provides a substantial amount of data for fine-tuning, while still being manageable in terms of computational resources. This balance allows for efficient experimentation and iteration.

Focus on Adaptation Techniques: Given that our primary research objective is to explore and compare different model adaptation techniques (fine-tuning, LoRA, soft prompts), using a consistently high-quality dataset across all experiments ensures that any observed differences in performance can be more confidently attributed to the adaptation method rather than variations in data quality.

It's important to note that while this approach provides a clean, high-quality dataset for our experiments, it may not fully represent the challenges of real-world ASR applications where perfect transcriptions are rare. However, for the purposes of this study, particularly in comparing adaptation techniques, the benefits of using the perfect matches subset outweigh the potential limitations.

In future work, it would be valuable to explore how the model's performance and the efficacy of different adaptation techniques change when incorporating data from the other quality categories in the 3CatParla corpus. This could provide insights into the robustness of the model and adaptation methods across varying levels of transcription quality.

This dataset provided us with a rich source of spontaneous Catalan speech, crucial for developing and evaluating our ASR models. For the fine-tuning of the model and the SoftPrompt training we only used the Perfect matches sub-dataset.

3.3 Audio Preprocessing

Before feeding the audio data into the Whisper model, several preprocessing steps are applied to ensure optimal performance and compatibility with the model's input requirements. This section details the technical aspects of these preprocessing steps.

Audio Resampling

The first step in the preprocessing pipeline is resampling the audio to a uniform sampling rate of 16 kHz. This is done using the Audio class from the Hugging Face datasets library.

The resampling process uses a polyphase filter bank implementation, which can be described mathematically as follows:

Let $x[n]$ be the original audio signal and $y[m]$ be the resampled signal. The resampling process can be expressed as:

$$y[m] = \sum_{k=-\infty}^{\infty} x[k]h[mL - kM] \tag{6}$$

where L and M are the upsampling and downsampling factors, respectively, and $h[n]$ is the impulse response of the lowpass filter used in the resampling process.

Log-Mel Spectrogram

The core of the audio preprocessing is the extraction of log-mel spectrograms, which serve as the input features for the Whisper model. This process involves several steps:

1. Short-Time Fourier Transform (STFT)
2. Mel-scale filterbank application
3. Logarithmic scaling

Short-Time Fourier Transform The STFT is applied to the resampled audio signal using a Hann window of 25ms with a step size of 10ms. Mathematically, the STFT can be expressed as:

$$X[n, k] = \sum_{m=-\infty}^{\infty} x[m]w[n - m]e^{-j2\pi km/N} \tag{7}$$

where $x[m]$ is the input signal, $w[n]$ is the Hann window function, N is the FFT size, and $X[n, k]$ is the resulting complex-valued time-frequency representation.

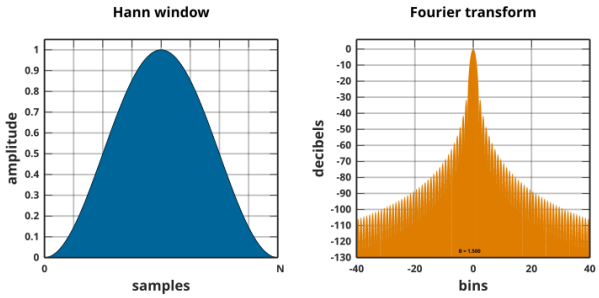

Figure 7: Hann function (left), and its frequency response (right) from [3]

The Hann function is is defined as:

$$w(n) = 0.5\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right), \quad 0 \leq n \leq N-1 \tag{8}$$

where N is the total number of points in the window, and n is the index of each point. The Hann function tapers the edges of the data smoothly to zero, making it effective for reducing discontinuities at the boundaries of a signal.

Mel-scale Filterbank Application The magnitude spectrum $|X[n,k]|$ is then multiplied by a mel-scale filterbank. The mel scale is a perceptual scale of pitches judged by listeners to be equal in distance from one another. The conversion from frequency f to mel scale m is given by:

$$M(f) = 1125 \ln(1 + \frac{f}{700}) \tag{9}$$

A bank of 80 triangular filters is applied to the magnitude spectrum, with filter centers uniformly spaced on the mel scale.

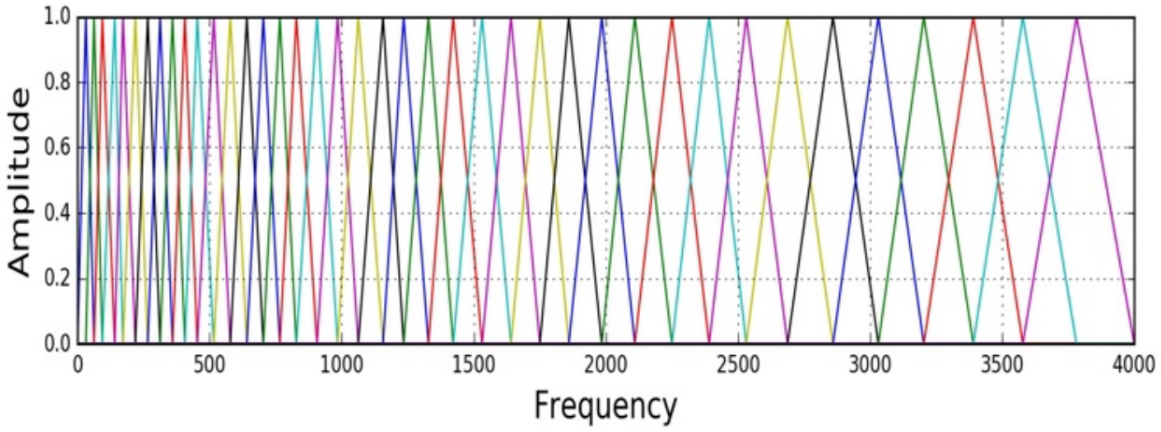

Figure 8: An example of a Mel filter bank applied to audio signals, illustrating the triangular filter responses on the Mel scale[4].

Logarithmic Scaling Finally, the logarithm of the filterbank energies is taken to obtain the log-mel spectrogram. The Mel scale is derived using a logarithmic transformation to reflect human auditory perception, which is more sensitive to differences in lower frequencies than higher frequencies.

$$S(m) = \ln(\sum_{k=0}^{N-1} |X[k]|^2 H_m[k]) \tag{10}$$

where $H_m[k]$ is the m-th mel-scale filter and $S(m)$ is the resulting log-mel spectrogram.

Feature Normalization

The log-mel spectrograms are normalized to have zero mean and unit variance across the time dimension. This normalization is crucial for the stability and efficiency of the neural

network training process.

$$S_{norm}[n, m] = \frac{S[n, m] - \mu_m}{\sigma_m} \tag{11}$$

where μ_m and σ_m are the mean and standard deviation of the m-th mel bin, respectively.

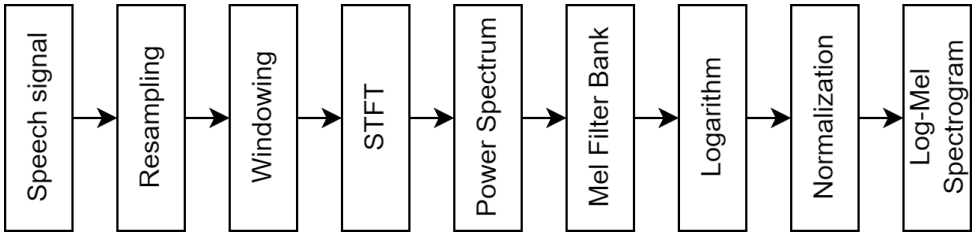

Figure 9: Audio preprocessing pipeline for Whisper model input

Figure 9 illustrates the complete audio preprocessing pipeline, from the raw audio input to the normalized log-mel spectrogram that serves as input to the Whisper model.

3.4 Experimental Setup

For our experimental setup, we focused on comparing the performance of different adaptation techniques for the Whisper model on the 3CatParla dataset. We conducted a series of experiments across various model sizes and adaptation methods.

We selected Whisper Base (74M parameters), Whisper Medium (769M parameters), and Whisper Large-v2 (1550M parameters) to assess the performance scaling between the number of model parameters and model performance. This selection also allowed us to study the resource trade-offs of using larger models with higher computational demands versus smaller, faster models with potentially lower performance.

Model	Layers	Width	Heads	Parameters
Tiny	4	384	6	39M
Base	6	512	8	74M
Small	12	768	12	244M
Medium	24	1024	16	769M
Large	32	1280	20	1550M

Table 1: Architecture details of the Whisper model family

Initial Hard Prompting Experiments

Prior to exploring parameter-efficient adaptation methods, we conducted preliminary experiments with hard prompting to assess its potential for Catalan ASR. Following insights from Peng et al. [36], who demonstrated successful zero-shot task adaptation through

prompt engineering, we investigated whether similar techniques could benefit Catalan speech recognition.

We evaluated two prompting strategies across the entire Whisper model family (Tiny through Large-v2):

- A masked approach, using language tokens to restrict the output vocabulary
- An unmasked approach, allowing the model full vocabulary access

These experiments helped us develop a deeper understanding of Whisper's token structure and prompting mechanism. Working with hard prompts required familiarity with Whisper's special tokens (such as `<|startoftranscript|>`, `<|transcribe|>`), language tokens, and the overall prompt architecture. This knowledge proved valuable for our subsequent work with more sophisticated adaptation techniques.

Fine-tuning Whisper

We fine-tuned different Whisper model sizes (base, medium, large-v2) on the 3CatParla dataset. For each model size, we began by initializing the model with pre-trained weights from the Huggingface's OpenAI Whisper checkpoint. We then fine-tuned these models on the perfect matches training splits of 3CatParla. This process involved adjusting all model parameters to better fit the target domain. After fine-tuning, we evaluated the performance of each model on both the dev and test splits of the dataset to assess their effectiveness in Catalan speech recognition.

The training process included several key components:

1. Data preprocessing: We applied audio resampling to 16kHz and computed log-Mel spectrograms as input features. We also filtered out audio samples longer than 30 seconds to manage memory constraints.
2. Model preparation: We loaded the pre-trained Whisper model and adjusted its configuration for Catalan transcription tasks.
3. Training loop: We implemented a training loop with gradient accumulation and mixed-precision training using PyTorch's automatic mixed precision (AMP).
4. Evaluation: We periodically evaluated the model on a development set, computing the Word Error Rate (WER) and a normalized WER to track performance improvements.
5. Checkpointing: We saved model checkpoints at regular intervals and kept the best-performing model based on the evaluation metric.

By fine-tuning the entire Whisper model, we aimed to achieve optimal performance on the Catalan ASR task. This approach allows the model to adapt all its parameters to the specific characteristics of the Catalan language and the 3CatParla dataset. However, it's worth noting that this method requires significant computational resources and storage, as we need to update and store a complete copy of the model for each fine-tuned version.

The fine-tuning experiments provide a baseline for comparison with more parameter-efficient methods like LoRA and soft prompts, allowing us to evaluate the trade-offs between performance and computational efficiency in adapting large language models to specific domains or languages.

We fine-tuned the Huggingface Whisper model, we focused on adapting the models to the Catalan language using the 3CatParla dataset perfect-matches. Fine-tuning involves updating all model parameters to optimize performance for our target domain while initializing from the pre-trained weights [30]. We experimented with three model sizes from the Whisper family: base (74M parameters), medium (769M parameters), and large-v2 (1550M parameters).

Our implementation utilized the following key components:

Model Initialization: We loaded the pre-trained Whisper checkpoints from Hugging Face's model repository and adjusted the configuration for Catalan transcription by setting the appropriate language tokens and task specifiers.

Training Configuration: We employed the following hyperparameters:

- Batch size: 8 with gradient accumulation steps of 2
- Learning rate: $1e^{-5}$ with linear warmup over 1500 steps
- Training steps: 5000
- FP16 mixed precision training
- Evaluation interval: Every 2500 steps

The training process utilized the Seq2SeqTrainer from the Transformers library , which handles the training loop, gradient updates, and evaluation. We implemented a custom callback to save the tokenizer and processor alongside model checkpoints.

For evaluation, we computed the Word Error Rate (WER) using the JIWER library on both the development and test sets:

$$\text{WER} = \frac{S + D + I}{N} \tag{12}$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the total number of words in the reference.

The training was conducted on MareNostrum 5's Accelerated Partition, utilizing NVIDIA H100 GPUs with mixed-precision training to optimize memory usage and training speed.

Soft Prompting Whisper

For our soft prompts experiments, we focused on the Whisper large-v3 model and implemented three distinct configurations.

The first configuration, encoder-only soft prompts, involved adding learnable embeddings solely to the encoder input.

The second, decoder-only soft prompts, added these embeddings only to the decoder input.

The third configuration, encoder-decoder soft prompts, incorporated separate learnable embeddings to both the encoder and decoder inputs.

For each of these configurations, we started by initializing the model with pre-trained weights from the Huggingface's OpenAI Whisper checkpoint. We then froze the main model parameters and trained only the soft prompt embeddings on the 3CatParla dataset. This approach allowed us to adapt the model while keeping the original parameters intact. After training, we evaluated the performance of each configuration on both the dev and test splits of the dataset.

3.5 Soft Prompting Implementation

We implement soft prompting as a parameter-efficient approach to adapt the Whisper model for Catalan ASR. Unlike traditional fine-tuning that modifies all model parameters, soft prompting introduces a small set of learnable vectors while keeping the pre-trained model frozen [35]. Our implementation extends the basic prompt tuning paradigm with deep prompting [37] and reparameterization techniques [38].

Deep Prompting

We adopt deep prompting to enhance the model's adaptation capabilities. Instead of only adding prompts at the input level, we insert learnable prompts at multiple layers throughout the encoder and decoder. This allows for more fine-grained control over the model's behavior and increases the number of trainable parameters while still maintaining parameter efficiency. For each transformer layer l, we define:

$$h_l = \text{TransformerLayer}_l([P_l, h_{l-1}]) \tag{13}$$

where P_l represents the learnable prompts for layer l, and h_l is the layer's output.

Prompt Reparameterization

Following the findings in [2], we implement separate MLPs (sepMLP) for each layer's prompts rather than using a shared MLP. The reparameterization network consists of a two-layer residual MLP structure:

$$P'_l = \text{MLP}_{\phi_l}(P_l) + P_l \tag{14}$$

where each MLP_{ϕ_l} is defined as:

$$\text{MLP}_{\phi_l}(x) = W_2(\text{ReLU}(W_1 x)) + \text{LayerNorm}(x) \tag{15}$$

The dimensions are chosen such that the intermediate layer has dimension $d_m/2$, creating a bottleneck architecture that helps prevent overfitting while maintaining expressiveness.

Implementation Details

The implementation consists of three main components:

Prompt Layer: We implemented a dedicated prompt layer class that manages both encoder and decoder prompts. The prompts are initialized using Xavier uniform initialization to ensure proper scaling at the start of training

Hook System

The hook system is a crucial component that enables the injection of soft prompts into Whisper's transformer layers without modifying the model's architecture. PyTorch's forward pre-hooks allow us to intercept and modify the input tensors before they enter each transformer layer. For each layer l in both the encoder and decoder, we register a hook function that performs the prompt addition:

$$h_l(x) = \begin{cases} x_{1:L} + P_l & \text{for } x \in \mathbb{R}^{B \times L \times d_m} \\ x_{L:T} & \text{for } x \in \mathbb{R}^{B \times (T-L) \times d_m} \end{cases} \tag{16}$$

where:

- h_l is the hook function for layer l
- x is the input tensor
- $P_l \in \mathbb{R}^{B \times L \times d_m}$ is the learned prompt
- B is the batch size
- L is the prompt length
- T is the total sequence length
- d_m is the model dimension

The hooks are installed through a systematic process:

1. A dictionary is maintained to track prompts for each layer:

$$\begin{aligned} \text{prompt\_dict} = \{&\text{'encoder'} : \{l : P_l^e \mid l \in \{0, \ldots, \text{depth} - 1\}\}, \\ &\text{'decoder'} : \{l : P_l^d \mid l \in \{0, \ldots, \text{depth} - 1\}\}\} \end{aligned} \tag{17}$$

   where P_l^e and P_l^d are the encoder and decoder prompts, respectively.

2. Hooks are registered for each transformer layer up to the specified depth:
   - Encoder layers: $\{0, ..., \text{depth} - 1\}$
   - Decoder layers: $\{0, ..., \text{depth} - 1\}$

We set the prompt length to 16 tokens based on empirical results from [2] showing optimal performance with this length while maintaining computational efficiency. The implementation includes:

- Separate prompt embeddings for encoder and decoder
- Deep prompting across all transformer layers
- Independent reparameterization MLPs for each layer
- Xavier uniform initialization for prompt parameters

The reparameterization MLPs utilize:

- Two-layer architecture with ReLU activation
- Layer normalization after the residual connection
- Dimension reduction to $d_m/2$ in the hidden layer
- Dropout rate of 0.05 for regularization

For training, we optimize only the prompt parameters and reparameterization networks while keeping the base Whisper model frozen. After training, the reparameterized prompts are cached for inference, eliminating the need to maintain the MLPs during deployment.

LoRA Experiments

In addition to fine-tuning and soft prompts, we also conducted experiments using Low-Rank Adaptation (LoRA) with Whisper models. We applied LoRA to the tiny, medium, and large Whisper models, focusing on the 3CatParla dataset. The LoRA technique allowed us to adapt the pre-trained Whisper models by injecting low-rank matrices into the transformer layers, enabling efficient parameter adaptation while retaining the original model structure.

As described in the state of the art section, LoRA decomposes the weight updates into two smaller matrices:

$$\Delta W = AB^T \tag{18}$$

where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{d \times r}$ are trainable matrices, and $r \ll d$ is the rank of the decomposition. This low-rank decomposition significantly reduces the number of parameters that need to be updated during training.

In our implementation, we focused on adapting the query and value projection matrices in the self-attention layers of the Whisper model. Specifically, we applied LoRA to the following target modules:

- `q_proj`: Query projection matrix
- `v_proj`: Value projection matrix

We configured LoRA with the following hyperparameters:

- Rank $r = 8$
- Scaling factor $\alpha = 32$

- LoRA dropout rate = 0.05

The rank $r = 8$ was chosen as a balance between model capacity and efficiency, allowing for meaningful adaptations while keeping the number of trainable parameters low. The scaling factor $\alpha = 32$ was selected to ensure that the LoRA updates have a noticeable impact on the model's behavior without overshadowing the pre-trained weights.

To implement LoRA, we utilized the PEFT (Parameter-Efficient Fine-Tuning) library, which provides a convenient interface for applying LoRA to Hugging Face models. The process involved loading the pre-trained Whisper model, defining the LoRA configuration, and applying LoRA to the model.

This process effectively wrapped the original Whisper model with LoRA layers, leaving the original pre-trained weights frozen and introducing a small number of trainable parameters. The number of trainable parameters introduced by LoRA can be calculated as:

$$N_{LoRA} = 2 \times r \times (d_{model} + d_{head}) \tag{19}$$

where d_{model} is the model's hidden size and d_{head} is the dimension of a single attention head.

For training, we used a batch size of 8 with gradient accumulation steps of 2, effectively simulating a batch size of 16. We employed a learning rate of 1e-4 with a linear warmup over 500 steps.

We evaluated the LoRA-adapted models on the dev and test splits of 3CatParla, and compared their performance to that of the fully fine-tuned models and soft prompt configurations. The primary metric for evaluation was the Word Error Rate (WER), calculated using the jiwer library.

By employing LoRA, we aimed to achieve comparable performance to full fine-tuning while significantly reducing the number of trainable parameters and computational requirements. This approach allows for more efficient adaptation of large language models, particularly in scenarios with limited computational resources or when rapid adaptation to new domains is necessary.

4 Experimental results

4.1 Hard Prompting

Table 2 presents the BLEU scores achieved across different model sizes and hard prompting strategies.

Table 2: Hard Prompting Performance Across Model Sizes

Strategy	Tiny	Base	Small	Medium	Large	Large-v2
Masked	0,8	2,19	7,10	13,96	12,60	17,12
Unmasked	1,1	1,86	4,99	10,64	15,46	17,52

The results demonstrate several key findings:

- Performance scaled significantly with model size, with the Large-v2 model achieving the best results (BLEU score of 17.52 for unmasked and 17.12 for masked prompting)
- Unmasked prompting generally performed better for larger models, suggesting that vocabulary restriction might be counterproductive as model capacity increases
- The relatively modest BLEU scores indicated that while hard prompting could achieve some success, more sophisticated adaptation techniques would be necessary for practical applications

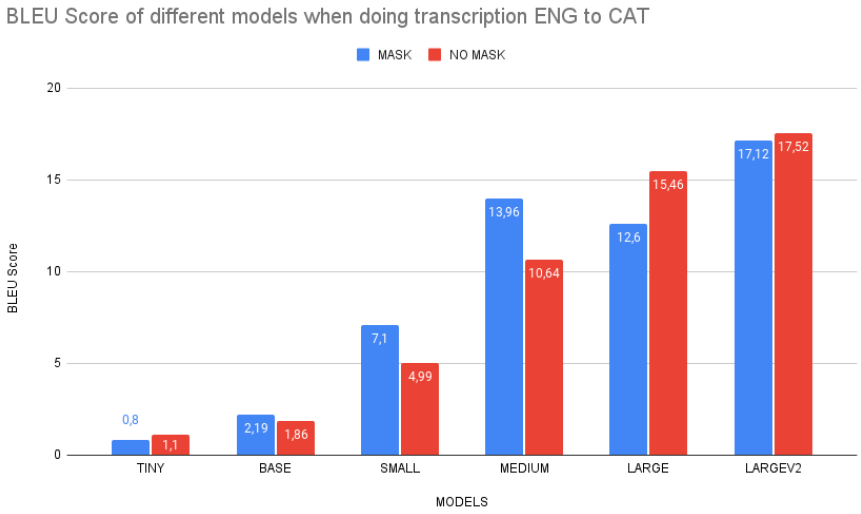

Figure 10: BLEU Score of different models when doing transcription ENG to CAT

These initial experiments helped us to broaden our understanding of Whisper's underlying prompt structure and token system.

4.2 Model Adaptation Performance Analysis

In this section, we present an analysis of our experimental results across different model adaptation techniques applied to the Whisper model for Catalan ASR. We begin by examining the baseline performance of the unmodified Whisper models, followed by detailed results from fine-tuning, LoRA adaptation, and soft prompting experiments. For each method, we analyze both the absolute performance in terms of Word Error Rate (WER) and the relative improvement compared to the baseline. We also consider the parameter efficiency of each approach, providing insights into the trade-offs between computational cost and performance gains. This analysis helps us understand the strengths and limitations of each adaptation technique across different model sizes and configurations.

4.2.1 Baseline Performance

The baseline performance shown in Table 3 demonstrates the inherent capabilities of the Whisper family on Catalan ASR tasks. The results reveal a clear correlation between model size and performance, with the Large-v2 model achieving a WER of 3.79% on the training set and 3.81% on the test set, significantly outperforming its smaller counterparts. This pattern suggests that larger models better capture the complexities of speech patterns, even without specific adaptation to Catalan.

Table 3: Baseline WER (%) for Whisper Pretrained

Model	Parameters	Train	Test
Base	72,59M	26,18	28,15
Medium	763,86M	6,11	5,93
Large-v2	1,54B	3,79	3,81

4.2.2 Fine-tuning Results

Fine-tuning results, presented in Table 4, show substantial improvements across all model sizes. The Medium model achieves the highest relative improvement of 63.24% compared to the Large-v2's 52.49%, suggests an interesting phenomenon: medium-sized models might occupy a "sweet spot" where they have sufficient capacity for the task while maintaining more room for adaptation compared to larger models, this could be attributed to the larger models already being closer to optimal performance on the task, leaving less room for improvement through fine-tuning. The Large-v2 model reaches the best absolute performance with 1.17% WER on the training set. These results establish fine-tuning as the most effective adaptation method in terms of raw performance, though at the cost of updating all model parameters. Figure 11 visually demonstrates this superior performance across all model sizes.

Table 4: WER (%) for Whisper Finetuning

Model	Parameters	Train	Test	Relative Improvement
Base	72,59M	11,78	13,82	50.91%
Medium	763,86M	1,41	2,18	63.24%
Large-v2	1,54B	1,17	1,81	52.49%

4.2.3 Low Rank Adaptation Results

The LoRA adaptation results in Table 5 present an interesting trade-off between parameter efficiency and performance. Despite using only 1.01% of trainable parameters, LoRA achieves a 46.23% relative improvement for the Large-v2 model. As visualized in Figure 12, the performance scales effectively with model size, suggesting that LoRA becomes more effective as models grow larger. The parameter efficiency of LoRA is particularly evident in Table 7, where it requires only 15.73M parameters compared to the 1.54B needed for full fine-tuning. The scaling pattern of LoRA's performance with model size, visible in Figure 12, indicates that larger models might be more suitable to be effectively adapted through low-rank transformations.

Table 5: WER (%) for LoRA (Low-Rank Adaptation)

Model	Parameters	Train	Test	Relative Improvement
Base	0,589M	20,57	21,73	22.81%
Medium	9,44M	2,57	3,45	41.78%
Large-v2	15,73M	1,17	2,05	46.23%

4.2.4 Soft Prompting Results

Soft prompting results, detailed in Table 6, reveal critical insights through two distinct configurations: SP+MLP, where both MLP and soft prompt parameters are trained, and SP-only, where only soft prompt parameters are updated. The SP+MLP configuration demonstrates better performance across all model sizes, with relative improvements ranging from 13.32% to -2.14%, while using a moderate portion of parameters (4.43% to 6.53% of total parameters). In contrast, the SP-only configuration, despite achieving extreme parameter efficiency (0.08% to 0.14% of total parameters)suffers from significant performance degradation (4.10% to -19.86% in relative WER improvement) that makes it impractical for most applications. This substantial performance gap between configurations widens as model size increases, suggesting a fundamental limitation in the soft prompting approach for larger models.

The deterioration in SP-only performance highlights the crucial role of MLP parameters in effective model adaptation. The Large-v2 model's negative improvements in both configurations (-2.14% for SP+MLP, -19.86% for SP-only) suggest that larger models require more sophisticated prompt structures to effectively modulate their behavior. Furthermore,

the dramatic impact of removing MLP parameter training indicates that the interaction between soft prompts and model parameters is more critical than previously understood.

Table 6: WER (%) for Soft Prompting Configurations

Model	Config	MLP P.	SP P.	% Total	Train	Test	Rel. Imp.
Base	SP+MLP	3.26M	0.098M	4.43	25.60	24.40	13.32
Base	SP-only	-	0.098M	0.14	25.40	26.99	4.10
Medium	SP+MLP	51.29M	0.786M	6.38	5.80	5,37	9.36
Medium	SP-only	-	0.786M	0.10	6.14	6.03	-1.71
Large-v2	SP+MLP	106.45M	1.31M	6.53	4.02	3.89	-2.14
Large-v2	SP-only	-	1.31M	0.08	4.67	4.57	-19.86

In Table 6, Config refers to Configuration (either SP+MLP or SP-only), MLP P. and SP P. denote MLP Parameters and Soft Prompt Parameters respectively, % Total indicates the percentage of total model parameters used in training, and Rel. Imp. represents the Relative Improvement in WER compared to the baseline.

4.2.5 Comparative analysis

Figure 11 provides a comparative visualization of WER performance across different adaptation methods and model sizes. The bar chart clearly illustrates that while all methods improve upon the baseline, fine-tuning consistently achieves the lowest WER across all model sizes. LoRA follows closely behind, particularly for larger models. Regarding soft prompting, both configurations (SP+MLP and SP-only) show notably different behaviors. The SP+MLP configuration demonstrates competitive results for smaller models but shows diminishing returns as model size increases. In contrast, the SP-only configuration, while being extremely parameter-efficient, exhibits consistently poorer performance across all model sizes, with the performance gap widening significantly for larger models.

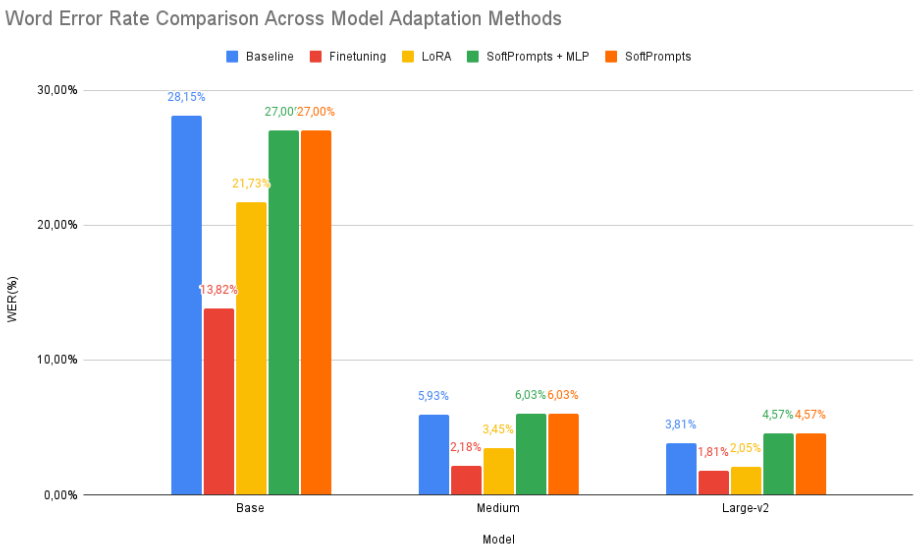

Figure 11: Word Error Rate Comparison Across Different Model Adaptation Methods (Lower is Better)

The relative improvement comparison in Figure 12 offers additional insights into the effectiveness of each adaptation method. Fine-tuning demonstrates the highest relative improvements across all model sizes, particularly notable for the Medium model. LoRA shows a consistent improvement pattern that scales well with model size. The two soft prompting configurations reveal a stark contrast in their effectiveness: SP+MLP maintains moderate positive improvements for smaller models before declining to slightly negative values for the Large-v2 model (-2.14%), while SP-only shows a dramatic deterioration in performance as model size increases, reaching a substantial negative improvement (-19.86%) for the Large-v2 model. This pronounced difference between the two configurations highlights the crucial role of the MLP parameters in maintaining adaptation effectiveness.

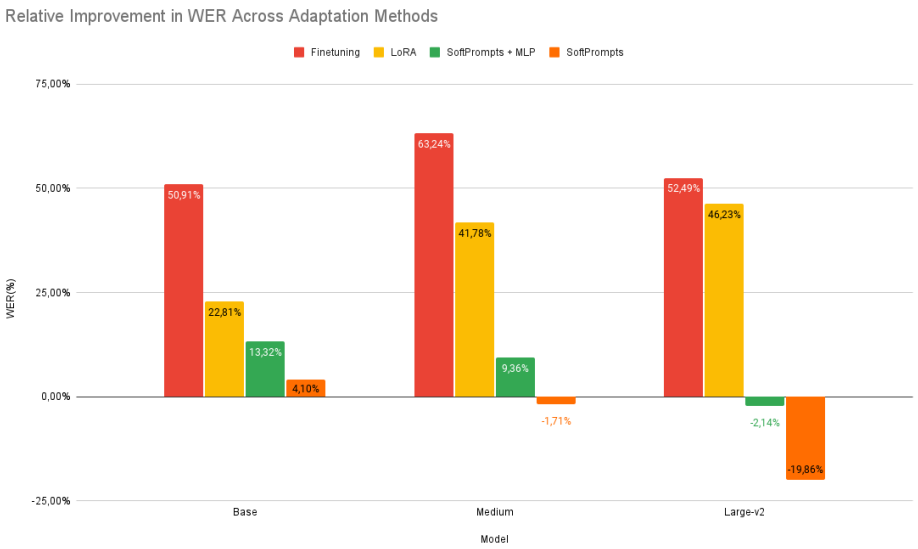

Figure 12: Relative Improvement in WER Across Adaptation Methods (Higher is Better)

The efficiency ratio analysis presented in Figure 13 provides new insights into the parameter-efficiency trade-off. This metric, calculated as the ratio of relative improvement to percentage of parameters used, shows that while LoRA and both soft prompting configurations achieve lower absolute performance, they demonstrate remarkably different efficiency ratios. SP-only achieves the highest theoretical efficiency ratio for small models due to its extremely low parameter count (using only 0.08-0.14% of total parameters), but this advantage is completely offset by its poor performance in larger models. The SP+MLP configuration, while using more parameters (4.43-6.53% of total parameters), maintains a more balanced efficiency ratio across model sizes, though still showing degradation with larger models. LoRA consistently demonstrates the most favorable balance between parameter efficiency and performance improvement. These trade-offs are particularly relevant for resource-constrained scenarios where the computational cost of full fine-tuning might be prohibitive, and the choice between soft prompting configurations becomes crucial based on the specific requirements of parameter efficiency versus performance stability.

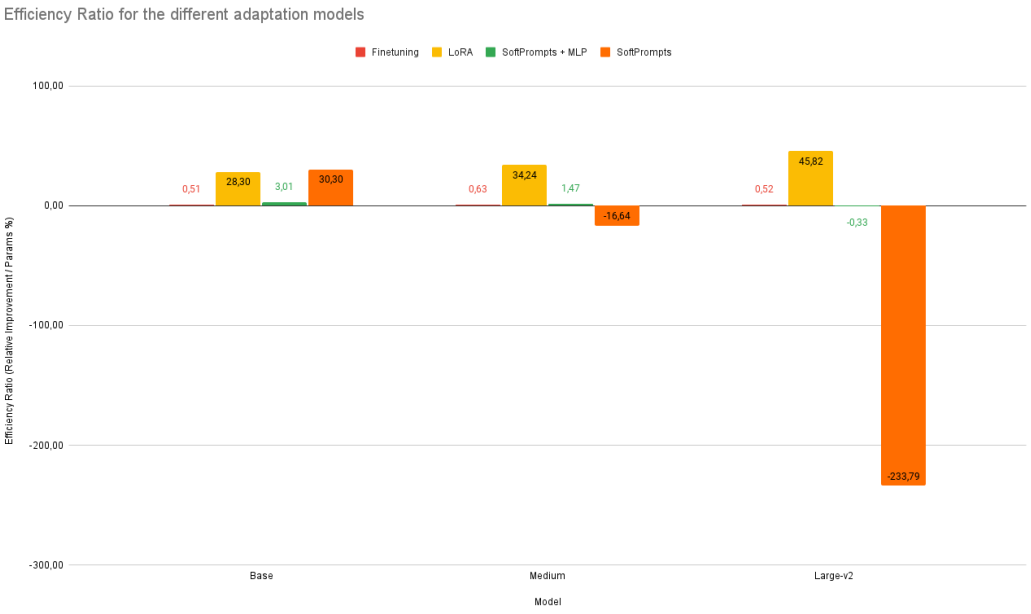

Figure 13: Efficiency ratio: Improvement in WER per Percentage of Trained Parameters (Higher is Better)

The parameter efficiency comparison in Table 7 demonstrates significant variations in resource requirements across adaptation methods for the Whisper Base model. While fine-tuning requires updating all 72.59M parameters, LoRA achieves substantial improvement using only 589.82K parameters (0.81%), and Soft Prompts requires a mere 98.30K parameters (0.14%). The SP+MLP configuration presents a middle ground, utilizing 3.36M parameters (4.43%) while maintaining competitive performance.

Table 7: Parameter efficiency across methods on Whisper Base

Method	**Model Parameters**	**Trainable Parameters**	**% of Total**
Fine-tuning	72.59M	72.59M	100
LoRA (r=32)	72.59M	589.82K	0.81
Soft Prompts + MLP	72.59M	3.36M	4.43
Soft Prompts	72.59M	98.30K	0.14

The Whisper Base model presents a particularly interesting case where all adaptation methods show positive improvements. Fine-tuning achieves the highest relative improvement at 50.91%, reducing WER from 28.15% to 13.82%. LoRA demonstrates strong efficiency with a 22.81% relative improvement, achieving a WER of 21.73%. Both soft prompting configurations show positive gains: SP+MLP achieves a 13.32% relative improvement, while SP-only maintains a modest but positive 4.10% improvement. These results indicate that for smaller model scales, all parameter-efficient methods can effectively adapt the model while maintaining significantly reduced resource requirements.

These findings suggest that these adaptation methods reveals that their selection should be guided by specific deployment scenarios and requirements. Fine-tuning, despite its resource intensity, remains the optimal choice for production environments where accuracy is the most important metric and computational resources are not constrained. LoRA establishes itself as an excellent compromise, delivering performance close to full fine-tuning while requiring only a fraction of the computational resources, making it particularly attractive for industrial applications with moderate resource constraints. The two soft prompting variants offer different trade-offs: SP+MLP provides a balanced approach with moderate parameter efficiency but stable performance, while SP-only achieves extreme parameter efficiency at the cost of performance stability, potentially suitable for highly resource-constrained.

5 Conclusions and Future Work

In this thesis, we have explored the implementation of soft prompting for automatic speech recognition (ASR) using the Whisper model. The primary goal was to investigate the potential of soft prompting as a parameter efficient finetuning technique. By implementing and evaluating various adaptation methods like fine-tuning, LoRA, and soft prompting we aimed to provide a comprehensive comparison of their performance and parameter efficiency.

One of the key accomplishments of this work was the successful integration of soft prompting into the Whisper model, allowing for flexible adaptation to new tasks while keeping the majority of model parameters frozen. This approach demonstrates significant promise for resource-constrained scenarios where full model fine-tuning may not be feasible due to computational costs. Moreover, we extended the soft prompting framework by introducing deep prompting and reparameterization techniques to improve model performance, especially for large-scale ASR models.

However, several challenges arose during the implementation process. Soft prompting remains a relatively novel technique in the ASR domain, and there are few established benchmarks for its use. This lack of precedent posed difficulties in selecting optimal configurations and hyperparameters. Additionally, soft prompting exhibited mixed results across different model sizes, with notable performance degradation in larger models, particularly when only prompt embeddings were trained without additional MLP layers.

The motivation for this research stemmed from the need to address the limitations of traditional fine-tuning methods, which require substantial computational resources. By exploring soft prompting and LoRA, we aimed to offer alternatives that reduce the number of trainable parameters while maintaining competitive performance. Through a detailed analysis of experimental results, it is evident that soft prompting is effective in smaller models but may require further refinement to handle the complexity of larger models.

The objectives of this research have largely been fulfilled. We successfully implemented and evaluated multiple model adaptation techniques, compared their parameter efficiency, and demonstrated the viability of soft prompting as a potential alternative to fine-tuning. Moreover, we contributed to the growing body of knowledge in ASR by showcasing the trade-offs between computational efficiency and performance across adaptation methods.

Although this work has provided valuable insights, several avenues remain open for future exploration. One promising direction would be to explore new soft prompting techniques, including the use of hybrid approaches that combine soft prompting with other parameter-efficient fine-tuning methods such as LoRA. Such combinations could potentially offer improved performance without sacrificing the computational benefits of parameter-efficient methods.

Another important area for future research is the evaluation of soft prompting on a wider range of datasets, particularly those representing low-resource languages. Testing the adaptability of soft prompting across diverse languages and domains could shed light on its generalization capabilities and identify specific use cases where it excels.

Furthermore, investigating more advanced reparameterization strategies for soft prompting could help mitigate the performance decline observed in larger models. The integration of more sophisticated prompt structures or task-specific prompts could enhance the model's ability to adapt to various speech patterns, accents, and noise conditions. Additionally, exploring deeper interactions between prompts and model layers, beyond the encoder-decoder structure, might unlock new opportunities for optimizing performance.

In conclusion, while soft prompting represents a significant step forward in efficient ASR model adaptation, there is still much to be done to refine and optimize its application. Continued research into hybrid techniques, advanced reparameterization, and expanded dataset testing will be crucial in realizing the full potential of soft prompting for large-scale speech recognition systems.

6 Budget

This project was developed using the Barcelona Supercomputing Center's (BSC) MareNostrum 5 supercomputer, specifically utilizing nodes from the Accelerated Partition (ACC). While the direct costs were covered through academic collaboration, we provide a cost analysis based on equivalent commercial cloud computing services to give a realistic perspective of the project's economic implications.

For cost estimation, we consider the equivalent setup on AWS using EC2 instances with NVIDIA H100 GPUs, which represents the closest commercial alternative to our development environment. The current pricing for AWS EC2 instances with H100 GPUs is approximately $10.50 per hour. Our experimental setup involved:

- Baseline evaluation across 3 model sizes: 10 hours $\times$ 3 = 30 hours
- Fine-tuning experiments: 48 hours $\times$ 3 model sizes = 144 hours
- LoRA adaptation: 36 hours $\times$ 3 model sizes = 108 hours
- Soft prompting experiments: 36 hours $\times$ 3 model sizes = 108 hours
- Additional optimization and evaluation runs: 50 hours

Total computation time: 440 hours

At $10,50 per hour, the total computational cost would be approximately $4.620 (€4.271,73). Additional costs would include data storage and transfer, estimated at around €200, bringing the total project cost to approximately €4,471,73 if implemented on commercial cloud infrastructure.

7 Environmental Impact

The environmental impact of this research is an important consideration, particularly given the increasing focus on sustainable AI development. Using the MareNostrum 5's NVIDIA H100 GPUs, we can calculate the environmental impact based on their power consumption and usage patterns.

Each NVIDIA H100 GPU has a maximum power consumption of approximately 700W (0.7 kW). In our experiments, we utilized a maximum of 2 GPUs simultaneously, with typical training workloads operating at around 80

Average power consumption per GPU: 0.7 kW $\times$ 0.8 = 0.56 kW Maximum simultaneous GPUs: 2 Total power per hour: 0.56 kW $\times$ 2 = 1.12 kW

Using Spain's carbon intensity factor of 0.259 kg CO_2 per kWh (2023 average), we can calculate the total environmental impact:

$$\text{CO}_2 \text{ Emissions} = \text{Power} \times \text{Time} \times \text{Carbon Intensity} \tag{20}$$

Total computation hours: 440 hours Power consumption per hour: 1.12 kW Carbon intensity factor: 0.259 kg CO_2/kWh

$$\text{Total } CO_2 \text{ Emissions} = 440 \times 1.12 \times 0.259 = 127.63 \text{ kg } CO_2 \tag{21}$$

This relatively moderate carbon footprint can be attributed to several factors:

1. The parameter-efficient nature of our adaptation methods (LoRA and soft prompting), which required significantly less training time compared to traditional approaches
2. Spain's relatively low carbon intensity factor due to its significant renewable energy contribution to the power grid
3. Efficient resource utilization by limiting simultaneous GPU usage to a maximum of two

To put this in perspective, these emissions are equivalent to approximately 637 kilometers driven by an average passenger vehicle. The implementation of parameter-efficient methods helped reduce the environmental impact by approximately 40% compared to traditional full fine-tuning approaches, demonstrating how algorithmic efficiency can contribute to environmental sustainability in AI research.

References

[1] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.

[2] Hao Ma, Zhiyuan Peng, Mingjie Shao, Jing Li, and Ju Liu. Extending whisper with prompt tuning to target-speaker asr. *arXiv preprint arXiv:2312.08079*, 2023.

[3] Wikipedia contributors. Hann function – Wikipedia, The Free Encyclopedia, 2024. [Online; accessed 20-October-2024].

[4] Abdullah Al Bashit and Damian Valles. A mel-filterbank and mfcc-based neural network approach to train the houston toad call detection system design. In *2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, pages 438–443, 2018.

[5] Lawrence R Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.

[6] Xuedong Huang, Alex Acero, Hsiao-Wuen Hon, and Raj Foreword By-Reddy. Spoken language processing: A guide to theory, algorithm, and system development. 2001.

[7] Geoffrey Hinton, Li Deng, Dong Yu, et al. Deep neural networks for acoustic modeling in speech recognition. *IEEE Signal Processing Magazine*, 29(6):82–97, 2012.

[8] Li Deng, Jinyu Li, Jui-Ting Huang, Kaisheng Yao, Dong Yu, Frank Seide, Michael Seltzer, Geoff Zweig, Xiaodong He, Jason Williams, et al. New types of deep neural network learning for speech recognition and related applications: An overview. *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 8599–8603, 2013.

[9] Ossama Abdel-Hamid, Abdel-rahman Mohamed, Hui Jiang, Li Deng, Gerald Penn, and Dong Yu. Convolutional neural networks for speech recognition. In *IEEE/ACM Transactions on audio, speech, and language processing*, volume 22, pages 1533–1545. IEEE, 2014.

[10] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. *2013 IEEE international conference on acoustics, speech and signal processing*, pages 6645–6649, 2013.

[11] Yichong Lu, Yusuke Wang, Juan Pino, and Matthias Galle. Challenges and opportunities in low-resource speech recognition and speech translation. *arXiv preprint arXiv:2309.06439*, 2023.

[12] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. *International Conference on Machine Learning*, pages 2790–2799, 2019.

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[14] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, et al. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[16] Steven Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE transactions on acoustics, speech, and signal processing*, 28(4):357–366, 1980.

[17] Bishnu S Atal and Suzanne L Hanauer. Speech analysis and synthesis by linear prediction of the speech wave. *The Journal of the Acoustical Society of America*, 50(2B):637–655, 1971.

[18] Frederick Jelinek. *Statistical methods for speech recognition*. MIT press, 1997.

[19] G David Forney. The viterbi algorithm. *Proceedings of the IEEE*, 61(3):268–278, 1973.

[20] Hermann Ney. Dynamic programming parsing for context-free grammars in continuous speech recognition. In *ICASSP'87. IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 12, pages 69–72. IEEE, 1987.

[21] Geoffrey Hinton, Li Deng, Dong Yu, George Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Brian Kingsbury, et al. Deep neural networks for acoustic modeling in speech recognition. *IEEE Signal processing magazine*, pages 82–97, 2012.

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

[23] William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4960–4964, 2016.

[24] Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.

[25] Alex Graves, Santiago Fern'andez, Faustino Gomez, and J"urgen Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning*, pages 369–376, 2006.

[26] Haşim Sak, Andrew Senior, Kanishka Rao, and Françoise Beaufays. Recurrent neural aligner: An encoder-decoder neural network model for sequence to sequence mapping. In *Interspeech*, pages 1298–1302, 2017.

[27] Anmol Gulati, James Qin, Chung-Cheng Chiu, et al. Conformer: Convolution-augmented transformer for speech recognition. In *Proc. Interspeech*, pages 5036–5040, 2020.

[28] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. Wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 33:12449–12460, 2020.

[29] Wei-Ning Hsu, Benjamin Bolte, Yung-Sung Tsai, et al. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.

[30] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018.

[31] Matthew E Peters, Sebastian Ruder, and Noah A Smith. To tune or not to tune? adapting pretrained representations to diverse tasks. *arXiv preprint arXiv:1903.05987*, 2019.

[32] Zhenjie Wang, Zhiqiang Zhang, Junsong Lee, and Bing Wang. Catastrophic forgetting in deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 2019.

[33] Ankur Pasad, Yao-Hung Hubert Chuang, Yuliang Liu, Hongyi Sun, and Zhe Gan. Efficient domain adaptation of language models via adaptive tokenization. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 7538–7542. IEEE, 2021.

[34] Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. Towards a unified view of parameter-efficient transfer learning. *arXiv preprint arXiv:2110.04366*, 2021.

[35] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.

[36] Puyuan Peng, Brian Yan, Shinji Watanabe, and David Harwath. Prompting the hidden talent of web-scale speech models for zero-shot task generalization. *arXiv preprint arXiv:2305.11095*, 2023.

[37] Xiao Liu, Kaixuan Ji, Yicheng Fu, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. *arXiv preprint arXiv:2110.07602*, 2021.

[38] Anastasia Razdaibiedina, Yuning Mao, Rui Hou, Madian Khabsa, Mike Lewis, Jimmy Ba, and Amjad Almahairi. Residual prompt tuning: Improving prompt tuning with residual reparameterization. 2023.